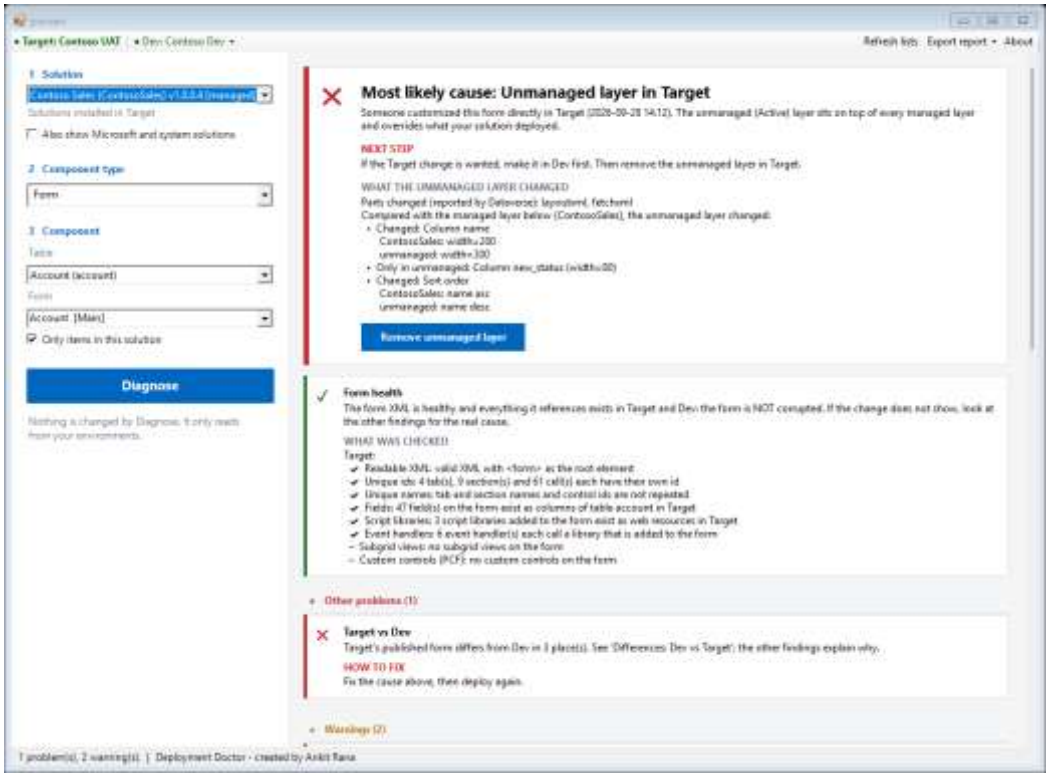
XrmToolBox tool · USER GUIDE

Deployment Doctor

Why doesn't my deployed change show in the target environment?

Check one component and get the cause and the fix, or compare a whole solution between Dev and Target in one go.



Sample data (a fictitious Contoso environment)

Version 1.5.2 · October 2026
Ankit Rana, Dynamics 365 / Power Platform Solutions Architect
linkedin.com/in/mrankitrana

Contents

1. What is Deployment Doctor? 3
2. The problem it solves 3
3. What it can check 4
What every check looks at 5
4. Before you start 5
5. Installing the tool 6
6. Check one item, step by step 7
Step 1: Connect to the Target environment 7
Step 2: Connect Dev 7
Step 3: Pick what to diagnose 7
Step 4: Click Diagnose 7
Step 5: Use a Fix, if one is offered 7
Step 6: Export the report 8
7. Reading the results 8
The verdict 8
Result levels 8
"Is it really corrupted?": the health check 9
The unmanaged layer 9
The Evidence section: User view and Developer view 9
8. Compare whole solution 10
9. Fixing problems from the tool 11
The "Before you continue" window 11
10. Sharing the result: Export report 12
Privacy 13
11. Tips and limitations 13
12. Frequently asked questions 13
Is it safe to run in Production? 13
Do I need to connect Dev? 13
Which account does a fix run as? 13
The verdict says "Deployment worked" but I still don't see the change. 14
Compare shows red for something I know is the same. 14
Why isn't the environment variable's current value a difference? 14
Does the report contain client data? 14
What if a fix is refused? 14
13. Getting help and giving feedback 14

1. What is Deployment Doctor?

Deployment Doctor is a free tool for XrmToolBox that answers the classic ALM question:

"I deployed the managed solution. Why doesn't my change show?"

It has two modes, switched in the toolbar:

- **Check one item**: pick the component that doesn't show your change. Deployment Doctor runs every relevant check in one click, shows the evidence, names the most likely cause and tells you how to fix it.
- **Compare whole solution**: pick the solution you deployed and see every component compared between Dev and Target. Red means the change did not arrive, orange means something needs a look, green means it is the same (the colours are used in the tool and in the exports).

Information	Detail
Tool	Deployment Doctor for XrmToolBox
Version covered	1.5.2
Works with	Forms, views, site maps, ribbons (command bars), classic workflows, business rules, actions, business process flows, cloud flows, environment variables, connection references, plug-in steps, plug-in assemblies and web resources
Access level	Diagnosis and compare are read-only. Fixes run only after your confirmation.
Install from	XrmToolBox Tool Library
Author	Ankit Rana
Feedback	github.com/ankitrana/XrmToolBox/issues

2. The problem it solves

A change made in Dev and shipped in a managed solution can fail to appear in UAT or Production for many different reasons. Each reason is checked in a different screen, and some aren't visible anywhere. Finding the cause by hand often takes hours of clicking between environments.

Cause	Applies to	What you see
Changes not published in Dev before export	Forms, views, web resources	The package simply doesn't contain the change.
Component not in the solution (for example a table added without it)	All	Import succeeds, but nothing changes.

Cause	Applies to	What you see
Unmanaged (Active) layer in Target	All	Someone edited it directly in Target. That overrides your deployment.
Another managed solution installed above yours	All	The other solution's version wins.
Staged upgrade never applied, or imported as Update	All	Removed items are still there.
Flow or workflow turned off after import; connection reference without a connection	Processes, connection references	Nothing runs.
Environment variable with no value in Target	Environment variables	Flows and apps read nothing.
Plug-in step disabled, or an older assembly version in Target	Plug-in steps and assemblies	The server logic never runs, or runs the old code.
Old copy with the same name still on	Processes	The old behaviour keeps happening.
Form or view inactive, missing from the app, not the default; form roles or order	Forms, views	Users open a different one.
Script not used by any form; browser cache	Web resources	The new code never runs.
Form XML damaged, or pointing at fields, scripts or views that don't exist	Forms, site maps, ribbons	"The form is corrupted."

Deployment Doctor checks all of these in one click.

3. What it can check

Component	What is compared and checked
Form	Tabs, sections, fields, field order, labels, visibility, scripts, event handlers. Form active, in the apps, security roles, form order. **Form health** (really corrupted, or only broken references?). Recreated forms with the same name.
View	Columns and widths, column order, filters, sort order, related tables. View active, default view, in the apps.
Site map	Areas, groups and sub areas (title, table, URL, icon, privilege, order). **Site map health**. Which apps use it.

Component	What is compared and checked
Ribbon (command bar)	Buttons added or hidden, commands and the scripts they run, rules, labels. **Ribbon health**.
Process / cloud flow	Workflow definition lines. For flows: triggers, steps and connection references. Turned on or off. Old copies with the same name.
Environment variable	Type and default value. Whether Target has any value. (Current values differ per environment by design, so they are shown, not compared.)
Connection reference	Connector, and whether it has a connection in Target.
Plug-in step	Message, stage, mode, execution order, plug-in class and assembly version, filtering columns. Enabled or disabled.
Plug-in assembly	Version, isolation mode and the DLL itself (size and fingerprint).
Web resource	Changed lines (JS, HTML, CSS, XML, SVG, RESX). For images the size and a hash. Which forms use it.

What every check looks at

- **Dev**: unpublished changes (an export takes the published version) and whether the component is in the solution.
- **Target**: whether the component is in the solution, and the solution versions in Dev and Target.
- **Solution layers**: the unmanaged (Active) layer on top, other managed solutions above yours, a staged upgrade that was never applied.
- **Content**: Dev vs Target vs your solution's layer, shown in readable form.

4. Before you start

- XrmToolBox installed on a Windows PC (www.xrmtoolbox.com).
- **Target connection.** The environment where the change doesn't show (UAT or Production). It is the main XrmToolBox connection.
- **Dev connection** (needed for Compare whole solution, and recommended for one item). The environment where the change was made. It is added as a second connection inside the tool and unlocks the most useful checks.
- **Permissions.** Reading is enough to diagnose and compare. To use a Fix, the connected account needs permission to make that change. Dataverse checks this, and if it refuses, nothing changes.

> **The tool never changes anything without your confirmation:** Diagnosing and comparing only read. A fix runs only when you click a Fix button and confirm it in a window that explains exactly what will change.

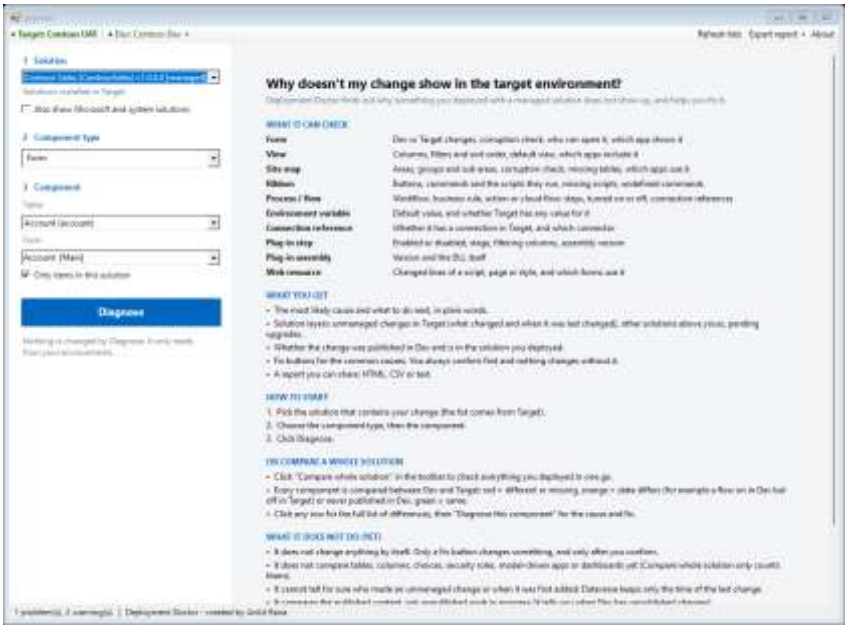
5. Installing the tool

1. Open XrmToolBox.
2. Open the **Tool Library**.
3. Search for **Deployment Doctor**.
4. Click **Install**, then restart XrmToolBox if it asks.

XrmToolBox also shows an update notice when a new version is published, with the release notes.

6. Check one item, step by step

The start screen lists what the tool can check, what you get, how to start, and what it does not do yet.



The start screen

Step 1: Connect to the Target environment

In XrmToolBox connect to the environment where the change doesn't show, then open **Deployment Doctor**. The toolbar shows **Target: <name>** in green when it is connected.

Step 2: Connect Dev

Open the **Dev** menu in the toolbar and choose **Connect Dev (source)...**, then pick your Dev connection. Only one Dev connection is kept; a new one replaces the old one.

Step 3: Pick what to diagnose

1. **1 Solution**: the solutions installed in Target. Tick *Also show Microsoft and system solutions* if you need those.
2. **2 Component type**: Form, View, Site map, Ribbon, Process / cloud flow, Environment variable, Connection reference, Plug-in step, Plug-in assembly or Web resource.
3. **3 Component**: the table (or prefix) and then the component. Type in the box to filter long lists. *Only items in this solution* is ticked by default.

Step 4: Click Diagnose

The right side shows the answer first, then the evidence. Nothing is changed by Diagnose.

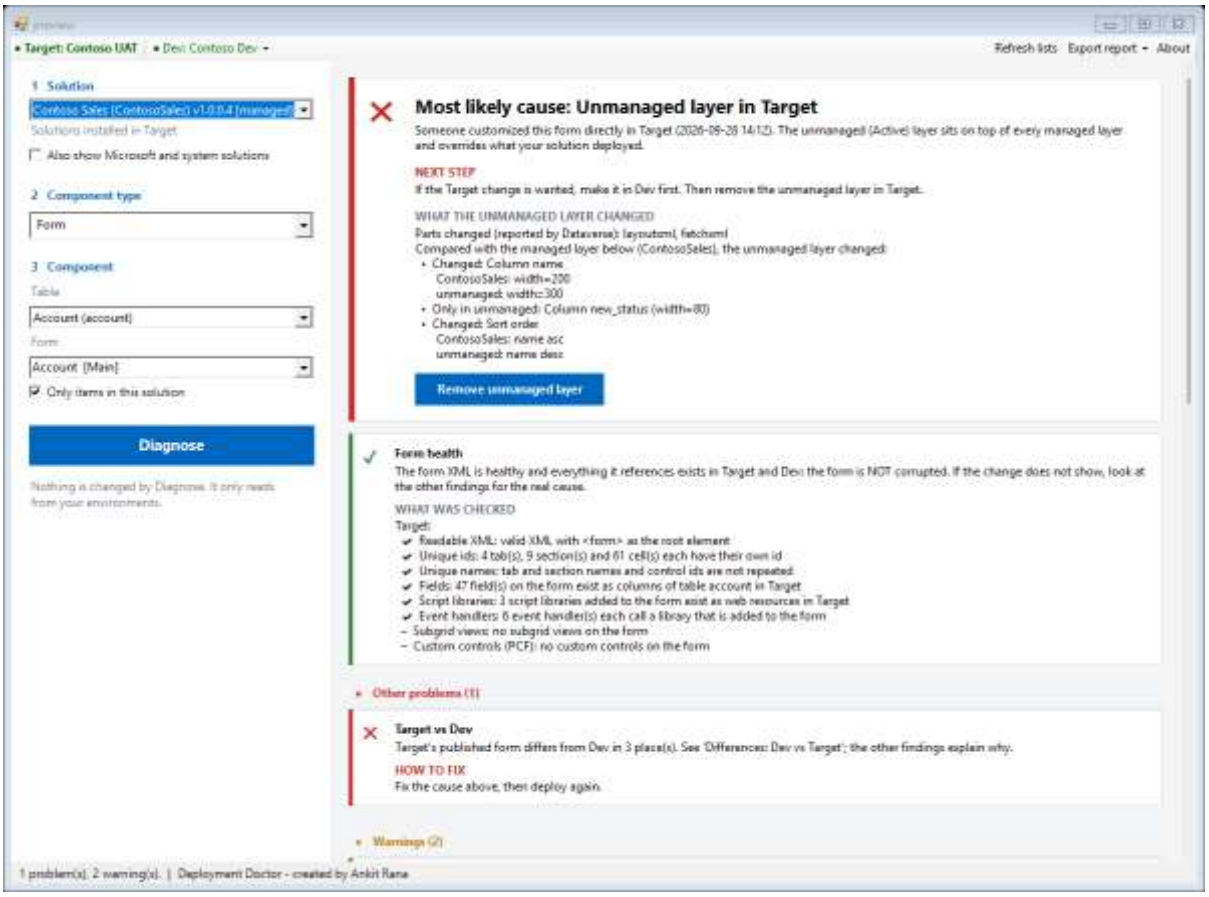
Step 5: Use a Fix, if one is offered

A **Fix** button appears next to a finding when the tool can change it for you. See section 9.

Step 6: Export the report

Use **Export report** in the toolbar to share the result. See section 10.

7. Reading the results



A diagnosis: the most likely cause first, then what was checked

The verdict

The big card at the top names the **most likely cause**, says what to do next, and shows the main fix button if there is one. If the only difference is the symptom ("Target vs Dev"), the card names the real cause when one was found.

Result levels

Level	Meaning
Problem (red)	Something is wrong and is probably why the change doesn't show.
Warning (orange)	Worth a look; it may explain what users see.
OK (green)	Checked and fine.
Info (grey)	Useful background.

Problems and warnings are shown as cards with a **How to fix** line. Passed checks and the evidence are collapsed until you open them; **Show all details** in the toolbar opens everything. Right-click any card and choose **Copy text** to paste it into a chat or ticket.

"Is it really corrupted?": the health check

For forms, site maps and ribbons the first card answers whether the item is really corrupted:

- **Damaged XML** (cannot be read, duplicate ids, wrong structure): it is really corrupted and needs repairing in Dev.
- **Healthy XML but broken references** (fields, scripts, views, quick view forms, controls, tables that don't exist in that environment): it is not corrupted, but it is broken. Add the missing items to the solution.
- **Healthy**: a green card says so.

Under the card, **What was checked** lists every test for each environment with a tick, a cross, a warning sign or a dash (nothing of that kind on the item), for example *Fields: 47 fields on the form exist as columns of table account in Target*. You can show this to a developer.

The unmanaged layer

If someone customized the component directly in Target, the verdict names the unmanaged layer. Under it, **What the unmanaged layer changed** lists:

- the parts Dataverse says changed, and what differs from the managed layer right below it;
- when the unmanaged layer was **last changed** (in your computer's local time);
- whether that is **older or newer than your last deployment**: older means it was already there and the deployment did not remove it, newer means it was made after the deployment.

> **Limit:** Dataverse keeps only the time a layer was last written, so the tool cannot tell when it was first added, and it cannot tell for sure who made the change. The component's own modified date is not used because every import changes it.

The Evidence section: User view and Developer view

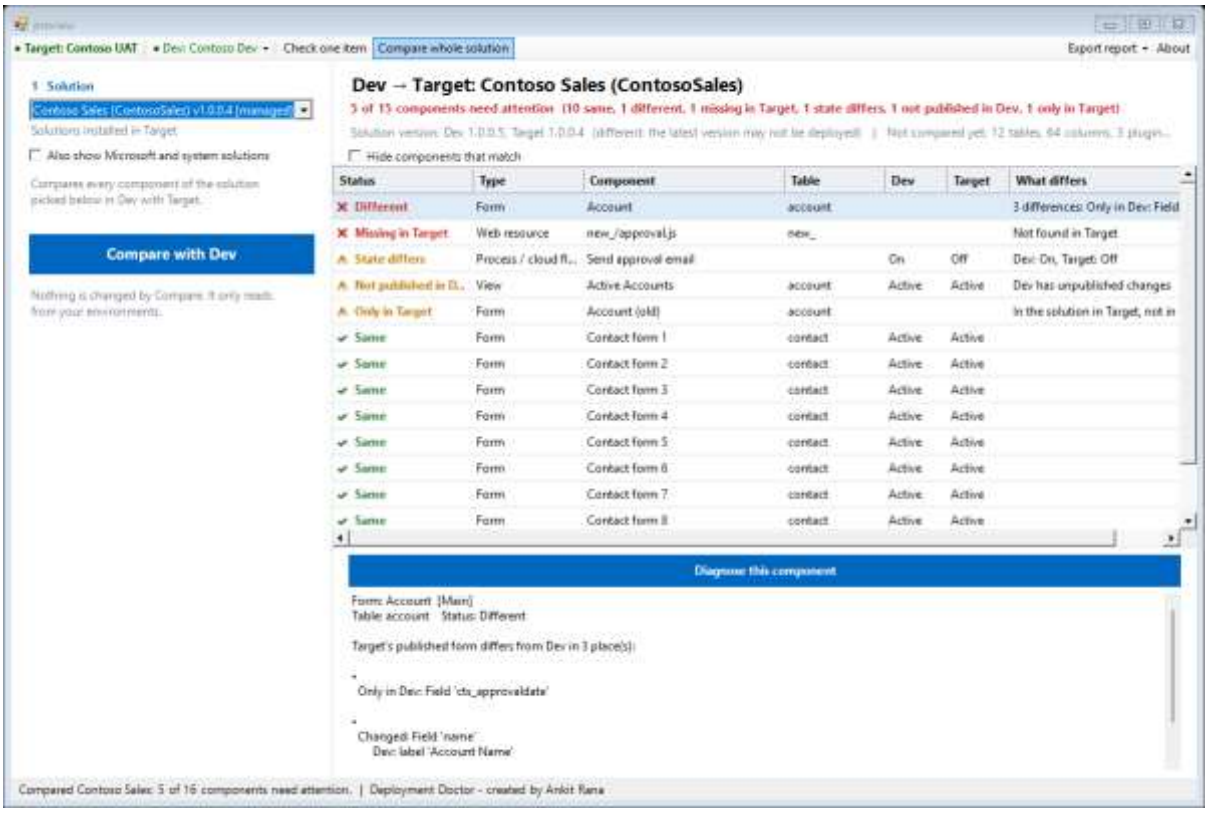
- **User view**: plain words. The solution layers as labelled cards (UNMANAGED, ABOVE YOURS, YOUR SOLUTION, BELOW YOURS) and a Dev vs Target table with *Missing in Target*, *Different* and *Only in Target*.
- **Developer view**: the detailed tabs: layers grid, differences as text, and the Target, Dev and layer content.

8. Compare whole solution

Use this after a deployment to check that everything arrived.

1. Connect Target (main connection) and Dev (second connection).
2. Click **Compare whole solution** in the toolbar.
3. Pick the solution on the left and click **Compare with Dev**.

Every component of the solution in Dev is found in Target and compared. A progress message shows where it is. A large solution can take a minute or more.



Compare whole solution (sample data)

Status	Meaning
Same	Content and state match.
Different	The published content differs. The details list exactly what.
Missing in Target	The component exists in the solution in Dev but not in Target: the import did not deliver it.
State differs	The content is the same but the state is not, for example a flow on in Dev and off in Target, an inactive form or view, a disabled plug-in step, a connection reference with no connection, an environment variable with no value.
Not published in Dev	Dev has unpublished changes. An export contains the published version, so those changes were not deployed.

Status	Meaning
Only in Target	Still in the solution in Target but no longer in Dev. An 'Update' import never deletes anything; import as 'Upgrade' or apply the staged upgrade.
Could not check	A read failed (for example no permission). The rest of the compare continues.

- The header shows the counts, the solution version in Dev and Target, and the component types that are **not compared yet** (tables, columns, plug-in registrations of other types and so on are only counted).
- Click a row for the full list of differences, plus a hint when Target has an unmanaged layer on it.
- **Diagnose this component** opens that component in the normal doctor, with the cause and the fix.
- Tick **Hide components that match** to see only what needs attention.
- **Export report** saves the compare as HTML, CSV or text (environment names only: no URLs and no component content).

> **Good to know:** A state such as "No value" or "No connection" in Target is shown in orange even when Dev has the same, because it stops the component from working. Environment variable values and connections differ between environments by design, so they are not treated as differences.

9. Fixing problems from the tool

When the tool can change something for you, a **Fix** button appears next to the finding:

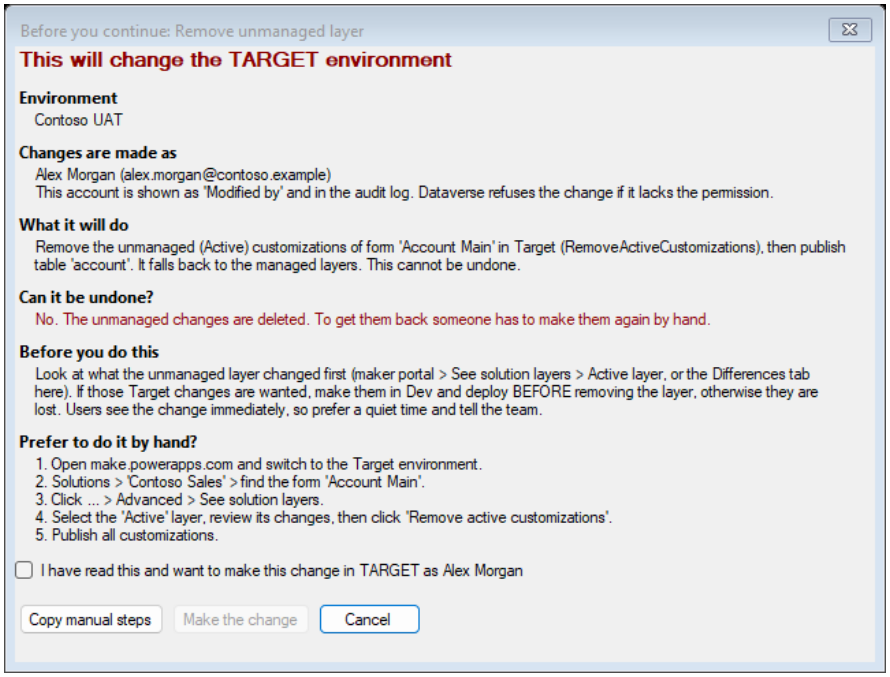
Fix	What it does	Where
Remove unmanaged layer	Removes the unmanaged (Active) customizations of a form, view, site map, web resource or process, then publishes where needed.	Target
Publish in Dev	Publishes the component in Dev.	Dev
Add to Dev solution	Adds the component to the unmanaged solution in Dev.	Dev
Turn on in Target	Activates a flow or workflow.	Target
Enable step in Target	Enables a disabled plug-in step.	Target

The tool does not remove the unmanaged layer of a ribbon, environment variable, connection reference or plug-in; for those it shows the manual steps.

The "Before you continue" window

Nothing changes on a direct click. A window first explains:

- **Environment**: which environment will change.
- **Changes are made as**: the user the connection runs as (Dataverse shows it as *Modified by*).
- **What it will do** and **Can it be undone?**
- **Before you do this**: what to check first.
- **Prefer to do it by hand?**: the manual steps, with a copy button.



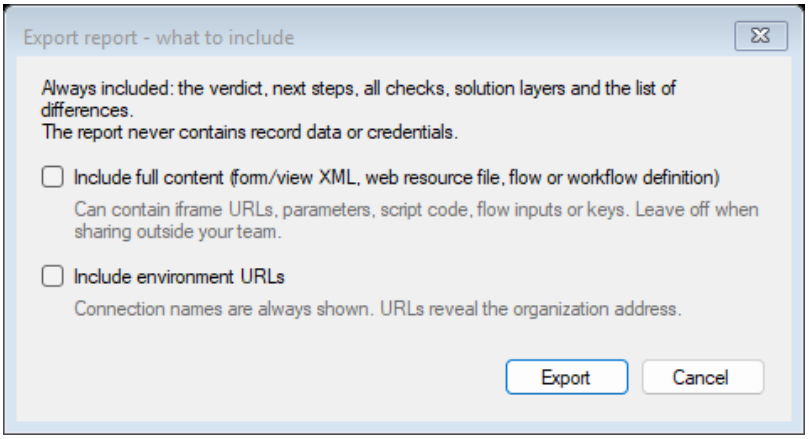
The confirmation window (sample data)

The **Make the change** button stays disabled until you tick "I have read this". Enter and Esc both mean Cancel.

10. Sharing the result: Export report

Export report (toolbar) saves the current result so anyone can read it:

- **HTML report**: to email or attach to a ticket.
- **CSV**: opens in Excel.
- **Copy as text**: for Teams or chat.



What may go into an HTML report

Privacy

Reports always contain the verdict, next steps, all checks (including the *What was checked* lists and the unmanaged-layer details), solution layers and the list of differences. They never contain record data or credentials. Form XML, file content and environment URLs are left out unless you tick them. The compare report contains environment names only.

11. Tips and limitations

- **Connect Dev** whenever you can. Without it, the Dev checks (publish state, solution membership, comparison) are skipped, and Compare whole solution is not available.
- It compares the **published** content, not work in progress. When Dev has unpublished changes, the tool says so.
- **Not compared yet** (they are only counted by Compare whole solution): tables, columns, choices, security roles, model-driven apps and dashboards.
- It **cannot tell for sure who made an unmanaged change or when it was first added**: Dataverse keeps only the time of the last change of a layer.
- It **cannot remove** the unmanaged layer of a ribbon, environment variable, connection reference or plug-in.
- Solution layers for components other than forms, views, web resources and processes are reported as Dataverse gives them; if a type shows no layers, use *See solution layers* in the maker portal.
- If the Dev connection points to the same environment as Target, the tool asks you to connect the real source environment.

12. Frequently asked questions

Is it safe to run in Production?

Diagnosing and comparing only read data. Nothing is changed unless you click a Fix button and confirm. Use a Fix in Production only at a quiet time and after reading the confirmation window.

Do I need to connect Dev?

Not to check one item, but it enables the most useful checks (publish state in Dev, solution membership in Dev, the Dev vs Target comparison). Compare whole solution needs it.

Which account does a fix run as?

The account of the XrmToolBox connection. The confirmation window shows it by name. It appears as *Modified by* in Dataverse.

The verdict says "Deployment worked" but I still don't see the change.

Target's published content is identical to Dev. Follow the next step in the card: usually a hard refresh (Ctrl+F5), checking which form or view the user opens, the user's security role, or whether a flow ran after the import.

Compare shows red for something I know is the same.

Click the row and read the differences: they are listed line by line. If a difference looks wrong, use **Diagnose this component** and send the report to feedback (section 13) so it can be fixed.

Why isn't the environment variable's current value a difference?

Current values are set per environment on purpose (for example a different URL in UAT). The tool shows them but only flags a variable that has no value at all.

Does the report contain client data?

No record data or credentials. Names of components and environments appear. Check the report before sharing it outside your team.

What if a fix is refused?

Dataverse checks the permission of the connected account. If it refuses, nothing changes, and the error is shown. Use the manual steps in the confirmation window instead.

13. Getting help and giving feedback

- Open an issue: github.com/ankitrana/XrmToolBox/issues (choose **Deployment Doctor**).
- How it works, for developers: DeploymentDoctor/docs/how-it-works.md in the same repository.
- Changes between versions: DeploymentDoctor/CHANGELOG.md.

License: MIT. Free to use, provided as is without warranty. Try it in a non-production environment first.